K8S-11: Multi-Tool Coexistence & Advanced Configuration

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 11 of 14 | **Created:** January 2026 | **Last Updated:** 10/13/2026

Running Dynatrace Alongside Other Monitoring Tools

Many organizations run multiple monitoring tools during migrations or for specialized use cases. This notebook covers patterns for running Dynatrace alongside tools like New Relic, Datadog, or Prometheus without conflicts.

Table of Contents

1. Coexistence Patterns
2. Opt-In Mode Configuration
3. Feature Flags Reference
4. Build Version Propagation
5. Injection Failure Policy
6. Log Monitoring Configuration
7. Complete Configuration Example

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring enabled
Kubernetes Cluster	Dynatrace Operator v1.0+ installed
Knowledge	Completed K8S-01 and K8S-02
Scenario	Running multiple monitoring tools

1. Coexistence Patterns

Understanding Your Options

When running Dynatrace alongside other APM tools, you have two primary approaches:

Approach	Description	Use Case
Infrastructure-Only	Omit `cloudNativeFullStack`	Dynatrace monitors K8s infrastructure; other tool handles APM
Selective APM (Opt-In)	Use namespace selectors	Different tools monitor different workloads

Infrastructure-Only Monitoring

If you omit `oneAgent.cloudNativeFullStack` entirely:

Included	Not Included
Kubernetes cluster health	Application-level APM
Workload metrics	Code-level tracing
Pod/container metrics	Distributed traces
Kubernetes events	OneAgent injection
ActiveGate capabilities	

```
# Infrastructure-only DynaKube (no cloudNativeFullStack)
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://your-tenant.live.dynatrace.com/api

  # Only ActiveGate for K8s monitoring - no OneAgent injection
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
    replicas: 2

  # Log monitoring still works without OneAgent
  logMonitoring: {}
```

This is appropriate when:
- Another tool (New Relic, Datadog) handles all APM
- You only need Dynatrace for K8s platform observability
- You want to avoid any agent conflicts

2. Opt-In Mode Configuration

Enable Selective Monitoring

There are two opt-in levels, and they behave differently. Pick one deliberately.

Level	How	What gets injected
Namespace opt-in	`namespaceSelector` inside the injection mode	Every pod in a matching namespace
Pod opt-in	`feature.dynatrace.com/automatic-injection: "false"` on the DynaKube	Only pods annotated `oneagent.dynatrace.com/inject: "true"`, in namespaces the DynaKube monitors

The feature flag does not opt namespaces in. The docs: with automatic injection off, *"Dynatrace Operator can be set to monitor namespaces without injecting into any Pods, so you can choose which Pods to monitor. Pods that should be injected have to be annotated with oneagent.dynatrace.com/inject: "true""*. Setting the flag and only labelling namespaces therefore injects nothing.

Namespace opt-in (most coexistence setups)

Step 1: Add a namespace selector so only labelled namespaces are monitored

```
spec:
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dt-monitoring: "true"
```

Step 2: Label the namespaces you want Dynatrace to monitor

```
kubectl label namespace checkout dt-monitoring=true
kubectl label namespace payment dt-monitoring=true

# Verify labels
kubectl get namespaces -l dt-monitoring=true
```

Pod opt-in (finer control)

Add the flag to the DynaKube, then annotate each pod template that should be injected:

```
# DynaKube
metadata:
  annotations:
    feature.dynatrace.com/automatic-injection: "false"
---
# Workload pod template
spec:
  template:
    metadata:
      annotations:
        oneagent.dynatrace.com/inject: "true"
```

Explicit Exclusions (Belt-and-Suspenders)

For extra safety, explicitly exclude other monitoring tool namespaces:

```
spec:
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dt-monitoring: "true"
        matchExpressions:
          # Explicitly exclude other monitoring namespaces
          - key: kubernetes.io/metadata.name
            operator: NotIn
            values:
              - newrelic
              - datadog
              - prometheus
              - kube-system
```

Namespace Labels Strategy

Label Value	Effect
`dt-monitoring: "true"`	Dynatrace monitors this namespace
No label	Namespace ignored by Dynatrace
`dt-monitoring: "false"`	Explicit opt-out (for documentation)

> **Sources:** DynaKube feature flags (DT docs) — *"Pods that should be injected have to be annotated with oneagent.dynatrace.com/inject: "true"".*

2a. OneAgent + OpenTelemetry Auto-Instrumentation Coexistence

A common multi-tool scenario in 2026 is running **OneAgent alongside OpenTelemetry auto-instrumentation** — typically when an organization uses OTel for open-standard language instrumentation while keeping OneAgent for infrastructure, network-zone and infrastructure-topology coverage. Both are supported ways to instrument a workload; the conflicts below are about two *injectors* reaching the same process, not about choosing one platform over the other.

The two injectors can conflict in several ways:

Conflict	Symptom	Mitigation
Double instrumentation of the same process	Duplicate spans, doubled metrics, increased CPU/memory overhead	Set `OTEL_INSTRUMENTATION_<lib>_ENABLED=false` for libraries OneAgent already covers, OR scope OTel injection by namespace selector
Two injectors in one pod	Pod stuck in `Init:CrashLoopBackOff`, or only one agent attaches	Pick one injector per namespace (or per workload)
Conflicting `LD_PRELOAD` / `JAVA_TOOL_OPTIONS`	Application starts but only one agent attaches; the other's env vars are clobbered	Keep OTel injection (the OpenTelemetry Operator's `instrumentation.opentelemetry.io/inject-*` annotations) and OneAgent injection on separate namespaces
Trace context conflicts	Spans appear in both backends but parent-child links are broken	Ensure both agents emit W3C trace context (`traceparent`); newer OneAgent versions and OTel SDKs both default to W3C

Analyzing the conflict surface

OTel auto-instrumentation reaches a Kubernetes pod in one of two ways: the **OpenTelemetry Operator**, a mutating webhook driven by `instrumentation.opentelemetry.io/inject-*` annotations, or the **OpenTelemetry injector** (github.com/open-telemetry/opentelemetry-injector), a shared library loaded through `LD_PRELOAD` or `/etc/ld.so.preload`. Both put an agent into the process at startup, which is exactly where OneAgent's code module goes. When evaluating coexistence, list:

- Which language runtimes are dual-injected (Java, Node.js and .NET agents differ)
- Which mechanism delivers the OTel agent, since `LD_PRELOAD` and webhook injection fail differently
- Which OneAgent mode is active: `cloudNativeFullStack` (host agent plus code modules) or `applicationMonitoring` (code modules only)

Recommended decision flow

1. **Inventory:** list every namespace running OTel auto-instrumentation today.
2. **Scope:** decide per namespace which auto-instrumentation *injector* each language runtime gets — OneAgent's code module or the OTel agent, not both in the same process. That is an injector rule, not a platform choice: OTel SDK spans in a OneAgent-instrumented process still reach Dynatrace through the Span Sensor or OTLP (see FAQ-03), and OneAgent keeps infrastructure coverage in OTel-managed namespaces (step 4).
3. **Configure DynaKube namespace selector** (Section 2 above) to exclude OTel-managed namespaces from OneAgent injection.
4. **Keep OneAgent for infra:** even in OTel-managed namespaces, OneAgent at the host/node level still provides Smartscape topology, network-zone routing, and host-level metrics — those don't conflict with in-pod instrumentation.

> See the **OTEL** series for OTel collector configuration. The OneAgent/OTel split is workload-specific — record it explicitly per language runtime and per namespace before instrumenting.

3. Feature Flags Reference

DynaKube Annotations

Feature flags are set as annotations on the DynaKube metadata:

```
metadata:
  annotations:
    feature.dynatrace.com/&lt;flag-name&gt;: "&lt;value&gt;"
```

Flags Used in This Notebook

Feature Flag	Values	Default	Purpose
`automatic-injection`	`true` / `false`	`true`	`false` = inject only pods annotated `oneagent.dynatrace.com/inject: "true"`

Feature Flag	Values	Default	Purpose
`injection-failure-policy`	`fail` / `silent`	`silent`	Pod startup behaviour when injection fails
`label-version-detection`	`true` / `false`	`false`	Propagate version labels to the injected OneAgent (§ 4)
`max-csi-mount-attempts`	integer	`10`	CSI driver mount attempts before the pod starts with a dummy volume, unmonitored
`max-csi-mount-timeout`	duration	`10m`	CSI driver mount timeout before the pod starts with a dummy volume, unmonitored

`k8s-app-enabled` is obsolete: the docs say it was *"Previously used to trigger the creation of the builtin:app-transition.kubernetes settings schema. The schema is no longer available on newer Dynatrace environments, where the Kubernetes app experience is enabled automatically."* Do not set it.

Recommended Configuration for Coexistence

```
metadata:
  annotations:
    # Detect version from Kubernetes labels
    feature.dynatrace.com/label-version-detection: "true"

    # Non-production: fail the pod if injection fails (production keeps the default,
silent)
    feature.dynatrace.com/injection-failure-policy: "fail"
```

Scope injection with a `namespaceSelector` (§ 2). Add `automatic-injection: "false"` only if you also annotate the pods to inject.

> **Sources:** DynaKube feature flags (DT docs) — *"Defines the maximum number of attempts for the Dynatrace Operator CSI driver to mount a volume. If this limit is reached, the Pod will start with a dummy volume, which will result in missing out on deep monitoring data."*

4. Build Version Propagation

Automatic Version Detection from Labels

Dynatrace can automatically detect application versions from Kubernetes labels.

Step 1: Enable the feature flag

```
metadata:
  annotations:
    feature.dynatrace.com/label-version-detection: "true"
```

> **Note:** You can enable this flag immediately. If labels are missing, Dynatrace simply won't have version metadata - no errors occur. When you add labels later, Dynatrace automatically picks them up.

Step 2: Add standard labels to your deployments

```
# In your application Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: checkout-api
  labels:
    app.kubernetes.io/name: checkout-api
    app.kubernetes.io/version: "1.2.3"
    app.kubernetes.io/component: api
    app.kubernetes.io/part-of: ecommerce
spec:
  template:
    metadata:
      labels:
        app.kubernetes.io/name: checkout-api
        app.kubernetes.io/version: "1.2.3"
```

Labels the flag reads

Pod label	Maps to
`app.kubernetes.io/version`	`DT_RELEASE_VERSION`
`app.kubernetes.io/part-of`	`DT_RELEASE_PRODUCT`
`dynatrace-release-stage`	`DT_RELEASE_STAGE`

"Only pod labels are detected, not workload (Deployment/StatefulSet) labels. To update a version, update the pod template and trigger a rollout. Patching labels directly with kubectl label on existing pods has no effect." The other `app.kubernetes.io/*` labels above are good Kubernetes practice but are not read by this flag. Build label propagation needs webhook injection, so it works with `applicationMonitoring` and `cloudNativeFullStack` only.

> **Sources:** Version detection methods (DT docs) — *"Only pod labels are detected, not workload*

> *(Deployment/StatefulSet) labels."*

5. Injection Failure Policy

Control Pod Behavior on Injection Failure

By default, if OneAgent injection fails, pods start anyway with silent failure. You can change this:

Policy	Behavior	Use Case
`silent` (default)	Pod starts without instrumentation	Production - availability first
`fail`	Pod fails to start	Non-prod - catch issues early

Enabling Fail Policy

```
metadata:
  annotations:
    feature.dynatrace.com/injection-failure-policy: "fail"
```

When to Use Each Policy

Environment	Recommended Policy	Rationale
Development	`fail`	Catch injection issues immediately
Staging	`fail`	Validate monitoring before production
Production	`silent`	Availability > perfect monitoring

Troubleshooting Injection Failures

If pods fail to start with `fail` policy, check:

```
# Check webhook logs
kubectl -n dynatrace logs -l app.kubernetes.io/component=webhook

# Check CSI driver logs
kubectl -n dynatrace logs -l app.kubernetes.io/component=csi-driver

# Check pod events
kubectl describe pod &lt;pod-name&gt; -n &lt;namespace&gt;
```

6. Log Monitoring Configuration

Understanding the Configuration Structure

> **Important:** Log monitoring has a split configuration:
> - `spec.logMonitoring` — enables the feature; its only field is the optional `ingestRuleMatchers`
> - `spec.templates.logMonitoring` — configures the Log module pods: image, resources, tolerations

Correct Configuration

```
spec:
  # Enable log monitoring ({} or with ingestRuleMatchers — nothing else)
  logMonitoring: {}

  # Resource configuration goes under templates
  templates:
    logMonitoring:
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 500m
          memory: 512Mi
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists
```

Common Mistakes

```
# WRONG - not fields of spec.logMonitoring
spec:
  logMonitoring:
    enabled: true          # Not a field
    resources: ...         # Belongs under templates.logMonitoring

# CORRECT
spec:
  logMonitoring: {}        # or ingestRuleMatchers, see below
  templates:
    logMonitoring:         # Resources go here
      resources: ...
```

Choosing Which Namespaces' Logs Are Collected

`spec.logMonitoring.ingestRuleMatchers` takes an `attribute` and a list of `values` for the initial log ingest rule — for example only the namespaces Dynatrace owns:

```
spec:
  logMonitoring:
    ingestRuleMatchers:
      - attribute: k8s.namespace.name
        values:
          - checkout
          - payment
```

The parameter reference marks it *"This field is immutable. Once set, it will no longer be updated."* Change the rules afterwards in **Settings → Log Monitoring → Log ingest rules**, not by editing the DynaKube.

> **Sources:** DynaKube parameters (DT docs) — *"This field is immutable. Once set, it will no longer be updated."*

```
// Which namespaces have OneAgent-instrumented pods (spans arriving from OneAgent)
// A pod that is injected but serves no traffic in the window does not appear.
fetch spans, from:-1h
| filter dt.openpipeline.source == "oneagent" and isNotNull(k8s.namespace.name)
| summarize {spans = count(), pods = countDistinctExact(k8s.pod.name)}, by:
{k8s.cluster.name, k8s.namespace.name}
| sort pods desc
| limit 30
```

```
// Check that version information is being detected
// DT_RELEASE_* values land on the process group instance as releasesVersion /
releasesProduct /
// releasesStage. Executed 10/02/2026: releasesVersion is a string such as
// "ReleaseVersionInfo{version='1.5.2', source=AGENT_REGISTRY, timestamp=0}".
fetch dt.entity.process_group_instance, from:-24h
| fieldsAdd releasesVersion, releasesProduct, releasesStage
| filter isNotNull(releasesVersion)
| summarize {instances = count()}, by:{releasesProduct, releasesVersion, releasesStage}
| sort instances desc
| limit 30
```

7. Complete Configuration Example

Production-Ready DynaKube with Coexistence

```yaml
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
  labels:
    dynatrace.com/created-by: dynatrace.kubernetes
  annotations:
    # Enable version detection from K8s labels
    feature.dynatrace.com/label-version-detection: "true"

    # Fail pod if injection fails (use 'silent' for production)
    feature.dynatrace.com/injection-failure-policy: "fail"
spec:
  apiUrl: https://your-tenant.live.dynatrace.com/api
  networkZone: production

  metadataEnrichment:
    enabled: true

  oneAgent:
    hostGroup: production
    cloudNativeFullStack:
      # Opt-in: Only monitor namespaces with this label
      namespaceSelector:
        matchLabels:
          dt-monitoring: "true"
        matchExpressions:
          # Explicitly exclude other monitoring namespaces
          - key: kubernetes.io/metadata.name
            operator: NotIn
            values:
              - newrelic
              - datadog
              - kube-system
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists

  activeGate:
    capabilities:
      - routing
      - kubernetes-monitoring
    resources:
      requests:
        cpu: 500m
```

```
        memory: 1Gi
      limits:
        cpu: 1000m
        memory: 2Gi
    replicas: 2

  templates:
    otelCollector:
      imageRef:
        repository: public.ecr.aws/dynatrace/dynatrace-otel-collector
        tag: "0.57.0"  # Example pin (released 09/24/2026) - never use 'latest'; check
the releases first
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 500m
          memory: 512Mi
    logMonitoring:
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 500m
          memory: 512Mi
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists

  # Enable log monitoring (resources configured above)
  logMonitoring: {}

  telemetryIngest:
    protocols:
      - otlp
    serviceName: telemetry-ingest
```

Summary

In this notebook, you learned:

- **Coexistence patterns** for running Dynatrace with other monitoring tools
- **Opt-in mode**: namespace selectors, and pod-level opt-in with the automatic-injection flag plus pod annotations
- **Feature flags reference** for DynaKube annotations
- **Build version propagation** from Kubernetes labels

- **Injection failure policy** for controlling pod startup behavior
- **Log monitoring configuration** with the split spec structure
- **Complete production configuration** for multi-tool environments

---

Next Steps

Next Notebook	Topic
K8S-12: Specialized Monitoring	NGINX Ingress, CSI Driver tuning
K8S-09: Troubleshooting	Diagnosing common issues

References

- DynaKube feature flags (DT docs)
- Annotate pods + namespaces (DT docs)
- DynaKube parameters (DT docs)
- Kubernetes recommended labels (kubernetes.io)
- Operator + DynaKube troubleshooting (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.